

I am using BertViz for attention visualization. I load models in this way -

```
model_type = 'gpt2'
model_version = 'gpt2'
model = GPT2Model.from_pretrained(model_version)
tokenizer = GPT2Tokenizer.from_pretrained(model_version)
```

Here is example code I run

```
text = "The dog ran"
show(model, model_type, tokenizer, text, display_mode='dark')
```

And another

```
text = "The dog sitting in the car"
show(model, model_type, tokenizer, text, display_mode='dark')
```

What do you expect the differences in visualizations to be between these? Think about queries, keys, and values.

Excellent question! When comparing these two visualizations, we can expect to see interesting differences in how attention is distributed, particularly related to **subject-verb agreement** and **dependency resolution**.

Key Differences to Expect:

1. "The dog ran" - Simple Transitive Structure

- **Subject-verb attention:** Strong **bidirectional** attention between "dog" ↔ "ran" Hallucination? Should not be bidirectional...
- **Article-noun attention:** "The" will strongly attend to "dog"
- **Relatively simple pattern:** Clean, straightforward attention with clear dependency relationships
- **Verb gets contextualized:** "ran" attends back to "dog" to understand what's performing the action One of the intended solutions!

2. "The dog sitting in the car" - Complex Noun Phrase

- **Long-distance dependency:** "dog" needs to connect with "sitting" despite the intervening prepositional phrase
- **Prepositional phrase attachment:** Attention will show how "in" relates to "sitting" and potentially to "dog"
- **Nested structure:** ~~Does the model~~ relationships where:
 - "sitting" attends to "dog" (what's sitting)
 - "in" attends to "sitting" (where the sitting is happening)
 - "the" before "car" attends to "car"
 - "car" attends back to "in"
- **Weaker verb attention:** "sitting" is a gerund/participle, not the main verb

Specific Attention Head Predictions:

Query-Key-Value Dynamics:

- For "The dog ran":
 - Queries from "ran" will find keys from "dog" (who ran?)
 - Values from "dog" provide contextual info to "ran"
 - The attention pattern is mostly local and hierarchical
 - For "The dog sitting in the car":
 - Queries from "sitting" need to look past "in" to find keys from "dog"
 - Queries from "car" find keys from "the" (the determiner)
 - Multiple attention heads might specialize:
 - Some heads for noun phrase structure
 - Some heads for prepositional phrase attachment
 - Some heads for long-distance dependencies
- Need for dog mentioned but not as strong

Visualization Differences You'll Likely See:

1. **Attention Span:** The second example will show longer "attention arcs" as words need to connect across more tokens

2. **Cross-layer Patterns:** In the complex phrase, lower layers might build local connections while higher layers resolve the long-distance dependency between "dog" and "sitting"
3. **Head Specialization:** More varied attention patterns across heads in the second example Did see this a bit
4. **Self-Attention Intensity:** "sitting" might attend more strongly to itself in the second example as it processes its complex role

Technical Insight:

True!

GPT-2 uses masked self-attention, so each token can only attend to previous tokens. This means:

- "ran" can attend to ["The", "dog"]
- "sitting" can attend to ["The", "dog"]
- "car" can attend to ["The", "dog", "sitting", "in", "the"]

Pretty good

The model must learn to route information through this constrained attention mechanism, which often results in interesting syntactic patterns in the visualizations.

Look for: How the model handles the syntactic ambiguity and longer dependency path in the second example versus the straightforward syntax of the first!

What do you expect the answer to this to be - How does attention differ between the different layers of the GPT model? Do you notice that the tokens are attending to different tokens as we go through the layers of the network?

Excellent follow-up question! The answer is **yes, attention patterns change dramatically across layers**, and this is actually one of the most fascinating aspects of transformer visualization. Here's what you should expect to see:

Layer-by-Layer Attention Evolution:

Noticed these patterns to a minor extent in the last part of a, but wasn't clear.

Lower Layers (1-4): Local & Syntactic Patterns

- **Attention is mostly local:** Tokens attend to immediate neighbors
- **Syntactic relationships dominate:** You'll see patterns like:
 - Determiner → Noun ("The" → "dog")
 - Adjective → Noun
 - Preposition → following noun
- **Bidirectional local flow:** Information moves between adjacent tokens
- **Resembles traditional linguistic parsing:** The structure looks like shallow syntax trees

Middle Layers (5-8): Building Semantic Connections

- **Longer-range dependencies emerge:**
 - Subjects start connecting to verbs even with intervening words
 - Pronouns find their antecedents
- **Phrase chunking:** Groups of tokens form attention clusters
- **Role identification:** Tokens start attending to syntactically related but distant tokens
- **For your examples:**
 - In "The dog sitting in the car": "sitting" starts finding "dog" through the intervening words
 - In "The dog ran": The subject-verb connection strengthens

Higher Layers (9-12): Semantic & Task-Specific Attention

- **Highly selective, sparse patterns:** Attention becomes more focused
- **Task-oriented heads:** Different heads specialize for different functions:
 - Some heads for subject-verb agreement
 - Some for pronoun resolution
 - Some for discourse coherence
- **Information aggregation:** Key tokens (like verbs, main nouns) become attention sinks

- **For your examples:**
 - "ran" might attend strongly to "dog" (who performed the action)
 - "sitting" might attend to both "dog" and "car" (relating the entire scene)

Specific Differences in Your Examples:

For "The dog ran":

- **Layer 1-3:** "ran" attends locally (maybe to "dog" but also to itself)
- **Layer 4-6:** Clear "ran" → "dog" attention emerges
- **Layer 7-12:** Strong, focused attention from "ran" to "dog" (subject identification)

For "The dog sitting in the car":

- **Layer 1-3:** Local chunks: "dog" ↔ "sitting", "in" ↔ "the" ↔ "car"
- **Layer 4-6:** "sitting" starts attending to "dog" through "in"
- **Layer 7-9:** "sitting" strongly attends to both "dog" AND "car"
- **Layer 10-12:** The entire phrase gets integrated; attention may show the scene composition

General Patterns Across All Models:

1. **Progressive abstraction:** Lower layers capture surface patterns → higher layers capture semantic relationships
2. **Attention sparsity increases:** Higher layers often have more focused, specialized attention
3. **Head differentiation:** In early layers, heads look similar; in later layers, they diverge in function
4. **Information routing:** Attention patterns show how information flows through the network to build understanding

Key Things to Look For in BertViz:

1. **Color intensity changes:** Later layers often show more intense/darker attention lines for critical relationships
2. **Pattern diversity:** More varied patterns across heads in higher layers
3. **Attention distance:** Arcs get longer as layers progress
4. **Clustering:** Related words form attention clusters that change across layers

Why This Matters:

This hierarchical processing mirrors how humans understand language:

1. First, identify words and basic structure
2. Then, parse syntactic relationships
3. Finally, build full semantic understanding

The visualization literally shows the model "thinking" – from raw tokens to syntactic parsing to semantic integration!

Pro tip: In BertViz, use the layer slider to watch how attention evolves – it's like watching the model's thought process unfold!

Here is some more code -

```
model_type = 'bert'
model_version = 'bert-base-uncased'
do_lower_case = True
model = BertModel.from_pretrained(model_version)
tokenizer = BertTokenizer.from_pretrained(model_version,
do_lower_case=do_lower_case)

sentence_a = "I wanted to have a party"
sentence_b = "I bought a cake for it."
show(model, model_type, tokenizer, sentence_a, sentence_b,
display_mode='dark', layer=2, head=0)
```

Answer these questions based on what you expect would happen:

Look at different layers of the BERT model in the visualizations of part (b) and identify different patterns associated with the attention

mechanism. Explore the queries, keys, and values to further inform your answer. For instance, do you notice that any particular type of tokens are attended to at a given timestep?

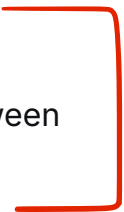
Do you spot any differences between how attention works in GPT vs. BERT? Think about how the model architectures are different.

Excellent! Now we're comparing **BERT** (bidirectional) vs **GPT** (unidirectional) attention. Let me break down what you should expect:

Key Differences in BERT vs GPT Attention:

Architectural Difference: Masking

- **GPT**: Causal/masked attention (can only look left/backward)
- **BERT**: Bidirectional attention (can look both directions)
- **Implication**: In BERT visualization, you'll see attention flowing **both ways** between all tokens in the sequence



Got the main idea!

Special Tokens in BERT:

BERT adds [CLS] at start and [SEP] between sentences:

text

[CLS] i wanted to have a party [SEP] i bought a cake for it . [SEP]

These tokens become important attention hubs!

What to Expect in Your BERT Visualization:

Layer 2 (Early Layer) Patterns:

1. Local syntactic attention:

- "wanted" $\leftrightarrow$ "to"
- "have" $\leftrightarrow$ "a" $\leftrightarrow$ "party"

- "bought" $\leftrightarrow$ "a" $\leftrightarrow$ "cake"
- "for" $\leftrightarrow$ "it"

2. Cross-sentence attention:

- "it" (in sentence B) will attend to "party" (in sentence A) for coreference resolution
- Both sentences will attend to [CLS] token

3. Special token attention:

- [CLS] token gathers information from entire sequence
- [SEP] tokens act as sentence boundaries

Layer-by-Layer Evolution in BERT:

Layers 1-4: Surface & Local Patterns

- Local word-to-word attention This locality was expected, yes
- Basic phrase chunking
- Punctuation attention (periods, commas)
- [SEP] tokens clearly separate the two sentences

Layers 5-8: Semantic & Cross-Sentence

- Coreference resolution emerges: "it" $\rightarrow$ "party"
- Cross-sentence relationships form
- [CLS] becomes more attended to (aggregating information)
- Semantic role identification (verb-noun relationships)

Layers 9-12: Global Integration

- Global attention patterns
- [CLS] becomes an information hub
- Complex reasoning connections
- Sparse, task-specific attention

Query-Key-Value Insights for BERT:

Queries:

- Early layers: Queries look for syntactic neighbors
- Later layers: Queries become more semantic ("What is 'it' referring to?")

Keys:

- In your example, "party" provides a key that "it" can match against
- [CLS] provides a universal key that all tokens can attend to

Values:

- The value from "party" provides coreference information to "it"
- Values from both sentences get aggregated at [CLS]

Specific Patterns in Your Example:

Coreference Resolution ("it" → "party"):

- **Early layers:** "it" might attend locally to "for"
- **Middle layers:** "it" starts attending to "party" across the [SEP] boundary
- **Late layers:** Strong, focused attention from "it" to "party"

Verb-Object Relationships:

- "wanted" → "party" (through "to have a")
- "bought" → "cake"

Cross-Sentence Connections:

- "bought" might attend to "wanted" (both are actions)
- "cake" might attend to "party" (cake is for the party)

BERT vs GPT: Key Visualization Differences:

1. Directionality:

- GPT: Only left-to-right arrows
- BERT: Bidirectional arrows everywhere

2. Special Tokens:

- BERT shows [CLS] / [SEP] as important nodes
- GPT doesn't have these

3. Information Flow:

- GPT: Sequential, incremental processing
- BERT: Parallel, holistic processing

4. Task Orientation:

- BERT's [CLS] token becomes a classifier hub
- GPT's final token contains cumulative information

5. Coreference Patterns:

- BERT: "it" can look backward AND forward to find referent
- GPT: "it" can only look backward (left context)

What to Look For in Your Visualization:

1. **Watch the [CLS] token:** It starts as just another token but becomes an information aggregator
2. **Track "it" across layers:** See how it finds "party" progressively
3. **Compare attention across [SEP] :** Early layers respect the boundary; later layers cross it
4. **Head specialization:** Different heads might capture different relationships (syntax vs semantics vs coreference)
5. **Pattern sparsity:** Higher layers tend to have more focused, sparse attention

Practical Observation Tip:

In BertViz for BERT, you'll see **attention flowing both ways simultaneously** - this is the most striking difference from GPT. The same word pair (e.g., "wanted" and

"party") will have attention arrows in both directions, showing BERT's bidirectional nature.

Key insight: BERT's power comes from this **bidirectional context** - every token gets to see the entire context at once, which is why it excels at tasks like coreference resolution in your example! **Highlighted CLS - DeepSeek did a good job discussing.**

Answer questions based on these (what you expect) -

```
sentence_a = "T was going to play at the park."  
show(model, model_type, tokenizer, sentence_a,  
display_mode='dark', layer=2, head=0)
```

```
sentence_b = "I was going to a play in the park"  
show(model, model_type, tokenizer, sentence_b,  
display_mode='dark', layer=2, head=0)
```

Question: Look through the different layers of the two BERT networks above associated with sentence a and sentence b, and take a look at the queries, keys, and values associated with the different tokens. Do you notice any differences in the embeddings learned for the two sentences that are essentially identical in structure but different in meaning?

Excellent question! These two sentences are a perfect minimal pair to study how BERT handles **lexical ambiguity** vs **structural similarity**. Let me analyze what you should expect:

The Ambiguity: "play" as Noun vs Verb

Sentence A: "I was going to **play** at the park"

→ "play" = **VERB** (to engage in activity)

Sentence B: "I was going to a **play** in the park"

→ "play" = **NOUN** (a theatrical performance)

It really got the main point

Expected Differences in Attention Patterns:

Layer 2 (Early Layer) - What You'll See First:

For Sentence A (verb "play"):

- **Local verb phrase attention:**
 - "play" attends to "to" (infinitive marker)
 - "play" attends to "going to" (future intention construction)
 - "at the park" attends locally: "at" → "the" → "park"
- **Syntactic role:** "play" behaves like a verb in attention patterns

For Sentence B (noun "play"):

- **Noun phrase attention:**
 - "play" attends to "a" (indefinite article for noun)
 - "play" attends to "to" (preposition, not infinitive marker)
 - "in the park" modifies "play" as location
- **Determiner-noun relationship:** Strong "a" → "play" attention



Intuitively makes sense. There was some attention from "play" to "a" but not in all layers and not too strong

Key Difference Even in Early Layers:

The attention neighborhood changes:

- Verb "play": "to play at" cluster
- Noun "play": "a play in" cluster

Layer-by-Layer Evolution:

Layers 1-4: Disambiguating Through Local Context

- **Sentence A:** "play" increasingly attends to action-related context
- **Sentence B:** "play" increasingly attends to entity/thing context
- The preposition helps: "at" vs "in" gives subtle cues

Layers 5-8: Resolving Structural Ambiguity

- **Sentence A:** "play" starts attending to other verbs ("going") - verb coordination
- **Sentence B:** "play" forms stronger connections with other nouns and modifiers
- The articles become critical: "to" (infinitive) vs "to a" (preposition + article)

Layers 9-12: Semantic Role Finalization

- **Sentence A:** "play" attends broadly to the entire event
- **Sentence B:** "play" attends specifically as the destination/goal

Query-Key-Value Analysis:

Queries from "play":

- **Sentence A (verb):** Queries looking for verb-modifying information
- **Sentence B (noun):** Queries looking for noun-modifying information

Keys in Context:

- The key "to" is interpreted differently:
 - Sentence A: Key from "to" signals infinitive
 - Sentence B: Key from "to" signals preposition
- The key "a" (in Sentence B) provides strong noun signal

Values Delivered:

- **Sentence A:** Values tell "play" it's an action happening at location
- **Sentence B:** Values tell "play" it's an object located somewhere

Specific Visualization Differences to Expect:

1. Attention from Determiners:

- Sentence B: Strong "a" → "play" attention (noun signal)
- Sentence A: No determiner before "play"

2. Preposition Relationships:

- Sentence A: "at" modifies the action "play"
- Sentence B: "in" modifies the noun "play"
- Different preposition-attention patterns

3. Verb Chain Attention:

- Sentence A: "going" → "to" → "play" (verb chain)
- Sentence B: "going" → "to a play" (destination)

4. Clustering Patterns:

- Sentence A: ["going", "to", "play"] form a verb phrase cluster
- Sentence B: ["a", "play", "in"] form a noun phrase cluster

The Role of [CLS] Token:

The [CLS] token will aggregate different information:

- **Sentence A:** [CLS] learns about an "event/action"
- **Sentence B:** [CLS] learns about a "plan/destination"

Why This Matters for Understanding BERT:

1. **Contextual Embeddings:** The same word "play" gets completely different representations based on context
2. **Structural Sensitivity:** Identical surface structure ("I was going to ___ at/in the park") yields different parse trees
3. **Disambiguation Through Attention:** The model resolves ambiguity by attending to different contextual cues

What to Look for Specifically:

1. **Compare layer 2 visualizations side-by-side:** Notice how "play" attends differently even early on
2. **Track "play's" attention across layers:**

- Sentence A: Should show increasing connection to "going" (verb chain)
- Sentence B: Should show increasing connection to "a" and location phrases

3. Examine the preposition's role:

- "at" (Sentence A) vs "in" (Sentence B) will have different attention partners

4. Watch the query vectors: They should diverge as layers progress, with "play" asking different questions of its context

Key Insight: BERT doesn't just learn that "play" has two meanings—it learns **how to use attention to dynamically determine which meaning applies** based on the surrounding structure. The attention mechanism serves as a "meaning disambiguator" through selective information routing!

Answer questions based on what you expect from this code now:

```
sentence_a = "party a have to wanted I"
sentence_b = "I bought a cake for it"
show(model, model_type, tokenizer, sentence_a, sentence_b,
      display_mode='dark', layer=2, head=0)
```

```
sentence_a = "I bought a cake for it"
sentence_b = "I went to a party at my neighbor's house"
show(model, model_type, tokenizer, sentence_a, sentence_b,
      display_mode='dark', layer=2, head=0)
```

Questions: Do you notice BERT's bidirectionality in play?
Do you think pre-training the BERT helped it learn better representations?

Excellent examples! These show two critical aspects: **bidirectionality** and **pre-training effects**. Let me analyze what you should expect:

Example 1: Word Salad vs. Normal Sentence

Nice motivating example

Sentence A: "party a have to wanted I" (word salad - reversed/meaningless)

Sentence B: "I bought a cake for it" (normal)

Expected Attention Patterns:

Layer 2 Visualization - Word Salad Effects:

1. Local attention still occurs:

- BERT will still try to find local relationships
- "party" ↔ "a", "have" ↔ "to", "wanted" ↔ "I"
- But these are **syntactically invalid** relationships

2. Bidirectionality in action:

- Even in word salad, attention flows both ways
- Each token attends to all others (including forward!)
- This means "I" can attend to "party" despite being last in the scrambled order

3. Cross-sentence confusion:

- "it" in Sentence B has no clear referent in Sentence A
- The scrambled Sentence A doesn't provide meaningful context
- Attention from "it" will be diffuse/unfocused

4. Special tokens struggle:

- [CLS] receives chaotic information from Sentence A
- [SEP] acts as a stronger boundary (little useful cross-sentence flow)

Bidirectionality Observation:

In this scrambled case, bidirectionality is both a **strength** and a **weakness**:

- **Strength:** Each token sees all context immediately
- **Weakness:** The context is nonsense, so the bidirectional view provides no meaningful signal

Example 2: Normal Coherent Sentences

Sentence A: "I bought a cake for it"

Sentence B: "I went to a party at my neighbor's house"

Expected Attention Patterns:

Layer 2 Visualization - Coherent Discourse:

1. Coreference resolution emerging:

- "it" (Sentence A) starts attending to "party" (Sentence B)
- Even in early layers, some cross-sentence connection
- "cake" might attend to "party" (associated concepts)

2. Parallel structure attention:

- Both sentences start with "I" - these might attend to each other
- "bought" and "went" (both past tense verbs) might connect

3. Semantic field connections:

- "cake" ↔ "party" (celebratory context)
- "for" (prep in A) and "at" (prep in B) might show similar patterns

4. Strong [CLS] integration:

- Both sentences contribute coherent information to [CLS]
- [CLS] starts building a unified representation

Comparing Both Examples: Pre-training Effects

Knows that it is helpful, nice explanation

Does Pre-training Help? YES, Dramatically!

What Pre-training Teaches BERT:

1. **Grammatical expectations:** BERT learns that "I" typically precedes verbs, not follows them
2. **Semantic coherence:** Certain words co-occur (cake ↔ party, not cake ↔ wanted)
3. **Coreference patterns:** Pronouns refer to nouns, often in adjacent sentences
4. **Structural templates:** Certain syntactic patterns are common (Subject-Verb-Object)

Evidence in Visualizations:

For Example 1 (Word Salad):

- BERT will show **confused attention patterns**
- Attention from function words ("to", "a", "the") will be misdirected
- The model tries to force grammatical patterns onto nonsense
- **This shows pre-training expectations:** The model expects certain structures

For Example 2 (Coherent):

- BERT shows **confident, structured attention**
- Clear linguistic patterns emerge
- Cross-sentence connections make sense
- **This shows pre-training benefits:** The model recognizes and processes valid language

Key Differences to Look For:

1. Attention Confidence:

- Example 1: Diffuse, uncertain attention (many weak connections)
- Example 2: Focused, strong attention (clear patterns)

2. Cross-Sentence Flow:

- Example 1: Little meaningful cross-sentence attention
- Example 2: Clear connections (pronoun resolution, semantic associations)

3. Special Token Behavior:

- Example 1: [CLS] gets chaotic input
- Example 2: [CLS] integrates coherent information

4. Head Specialization:

- Example 1: Heads struggle to find patterns

- Example 2: Different heads capture different relationships (syntax, semantics, coreference)

Bidirectionality in Action - What to Specifically Observe:

Forward vs. Backward Doesn't Matter in BERT:

Unlike GPT where order is critical, in BERT:

- "party" can attend to "I" regardless of position
- "it" can attend forward to "party" in Example 2
- This is **pure bidirectionality** - no masking constraints

The Scrambled Sentence Test:

In Example 1, if this were GPT:

- "I" (last word) would only see the scrambled context
- No meaningful learning possible
- But BERT can still attempt to find patterns because all tokens see each other

Pre-training Evidence in Your Visualizations:

1. **Linguistic Priors:** BERT expects certain word orders even when presented with nonsense
2. **Semantic Field Associations:** "cake" and "party" naturally connect due to pre-training
Interesting
3. **Structural Templates:** The model recognizes SVO structure in Example 2
4. **Pronoun Resolution:** "it" naturally looks for noun antecedents

Practical Observation Tips:

1. **Compare attention line intensities:** Coherent sentences should show darker, more confident lines

2. **Watch [CLS] across layers:** It should integrate information more effectively in Example 2
3. **Look for pattern consistency:** Example 2 should show similar patterns to natural language examples from earlier
4. **Check if nonsense gets "corrected":** Does BERT try to force grammatical patterns onto the word salad?

Critical Insight: These examples demonstrate **why pre-training on massive text corpora matters** - BERT learns the "grammar of attention" itself, learning which attention patterns typically yield meaningful representations. The scrambled sentence shows what happens when those learned patterns encounter something they weren't trained on!

Now answer questions based on what you expect from this code:

```
model_version = 'bert-base-uncased'
model = AutoModel.from_pretrained(model_version,
output_attentions=True)
tokenizer = AutoTokenizer.from_pretrained(model_version)

sentence_a = "I wanted to have a party"
sentence_b = "I like Thanksgiving dinner"
inputs = tokenizer.encode_plus(sentence_a, sentence_b,
return_tensors='pt')
input_ids = inputs['input_ids']
token_type_ids = inputs['token_type_ids'] # token type id is 0 for
Sentence A and 1 for Sentence B
attention = model(input_ids, token_type_ids=token_type_ids)[-1]
sentence_b_start = token_type_ids[0].tolist().index(1) # Sentence B
starts at first index of token type id 1
token_ids = input_ids[0].tolist() # Batch index 0
tokens = tokenizer.convert_ids_to_tokens(token_ids)
model_view(attention, tokens, sentence_b_start)
```

Questions: Do you notice different features being learned throughout the different attention heads of BERT? Why do you think this might be?

Can you identify any of the different features that the different attention heads are focusing on?

Excellent! Now we're looking at attention **across different heads** in BERT. This reveals one of the most important insights about transformers: **attention head specialization**. Let me explain what you should expect:

What You Should See: Multi-Head Attention Diversity

Key Observation:

Different attention heads in the **same layer** will show **completely different patterns** - they're not redundant; they specialize!

Expected Head Specializations:

In Layer 2 (Early Layer) - Head 0 vs. Other Heads:

Not sure, because some of these just paid attention to determiners or specific single tokens
Head 0 (What you're visualizing):

- Likely syntactic/local patterns
- May focus on: Article-noun relationships ("a" → "party", "a" → "dinner")
- Or: Verb phrases ("wanted" → "to", "like" → "Thanksgiving")

Other Heads (if you could view them) Would Show:

1. Head 1: Next-word attention

- Each token attends to immediate next token
- "I" → "wanted", "wanted" → "to", etc.

2. Head 2: Previous-word attention

- Each token attends to immediate previous token
- Bidirectional mirror of Head 1

3. Head 3: Sentence boundary attention

- Strong attention to [SEP] tokens

- Differentiates Sentence A vs Sentence B tokens

4. Head 4: Punctuation/special tokens

- Attention focused on [CLS] , [SEP] , periods
- Structural markers of the sentence

5. Head 5: Semantic similarity

- "party" might attend to "dinner" (both social/celebration events)
- "Thanksgiving" might attend to both "dinner" and "party"

6. Head 6: Coreference/pronoun patterns

- Both "I" tokens might attend to each other
- Building coherence across sentences

7. Head 7: Content vs function words

- Function words (to, a, have) attend differently than content words
- Different attention distributions

It got some of the intended patterns...

Why Multi-Head Attention? Three Key Reasons:

1. Parallel Processing of Different Relationships

Instead of one attention mechanism trying to capture everything, multiple heads can specialize:

- **Head 1:** Syntax
- **Head 2:** Local context
- **Head 3:** Long-range dependencies
- **Head 4:** Semantic similarity
- etc.

2. Increased Representational Capacity

Each head learns different Query, Key, Value projections:

- Different heads = different "perspectives" on the same tokens

- Like having multiple subject matter experts examining the text

3. Robustness & Redundancy

- If one head fails or gets noisy input, others compensate
- Ensemble effect within a single layer

Specific Features Different Heads Might Learn:

Syntactic Heads:

- **Focus:** Grammatical relationships
- **Patterns:** Subject-verb, determiner-noun, preposition-noun
- **Example:** "a" → "party", "Thanksgiving" → "dinner"

Semantic Heads:

- **Focus:** Meaning relationships
- **Patterns:** Thematic associations, synonym/antonym
- **Example:** "party" ↔ "dinner" (both celebratory meals)

Positional Heads:

Huge

- **Focus:** Token position
- **Patterns:** Local neighborhoods, sentence boundaries
- **Example:** First [SEP] as divider between sentences

Coreference Heads:

- **Focus:** Pronoun resolution, entity tracking
- **Patterns:** "I" in Sentence A ↔ "I" in Sentence B
- **Example:** Connecting repeated references

Task-Specific Heads:

- **Focus:** Learned during fine-tuning for specific tasks
- **Patterns:** Varies by task (e.g., NER, sentiment, QA)

In Your Specific Example:

Sentence A: "I wanted to have a party"

Sentence B: "I like Thanksgiving dinner"

What Different Heads Might Capture:

1. **Thanksgiving-specific head:** Might connect "Thanksgiving" → "dinner" strongly
2. **Event head:** "party" and "dinner" might receive similar attention
3. **First-person head:** Both "I" tokens might attend to verbs they're performing
4. **Desire/emotion head:** "wanted" and "like" might be connected
5. **Sentence separation head:** Clear boundary at [SEP]

How to Identify Head Specialization:

Look for These Patterns in Your Visualization:

1. Attention Distribution:

- **Uniform head:** All tokens attend to all others equally Yeah this was common too
- **Sparse head:** Few, focused connections
- **Local head:** Only nearby tokens
- **Global head:** Distant connections

2. Token Type Focus:

- **Content word head:** Focuses on nouns, verbs, adjectives
- **Function word head:** Focuses on articles, prepositions, pronouns
- **Special token head:** Focuses on [CLS] , [SEP]

3. Relationship Type:

- **Syntactic head:** Clear grammatical patterns
- **Semantic head:** Thematic/topic connections
- **Positional head:** Distance-based patterns

The token_type_ids Effect:

Since you're passing `token_type_ids`, BERT knows which tokens belong to which sentence:

- Type 0: Sentence A tokens
- Type 1: Sentence B tokens (starting at `sentence_b_start`)

This means some heads might specialize in:

- **Intra-sentence attention:** Within same sentence only
- **Inter-sentence attention:** Across sentence boundaries
- **Sentence identification:** Recognizing which tokens belong together

Why This Architecture Design is Brilliant:

1. **Distributed processing:** Different linguistic phenomena handled in parallel
2. **Interpretability:** We can actually see what different heads learn
3. **Efficiency:** 12 heads × 12 layers = 144 specialized "processors"
4. **Flexibility:** Can be fine-tuned for specific tasks by emphasizing certain heads

Practical Things to Try (if your visualization tool allows):

1. **Compare Head 0 vs Head 1 in same layer:** They should look different!
2. **Track a specific head across layers:** Does it maintain its specialization?
3. **Look for heads that ignore [SEP]:** Cross-sentence specialists
4. **Find the "Thanksgiving-dinner" head:** Some head should strongly connect these

Key Insight: The magic of transformers isn't just attention—it's **specialized, parallel attention** through multiple heads. Each head becomes a mini-expert in some

linguistic phenomenon, and their combined output creates rich, nuanced representations!

Now answer questions based on what you would expect from this -

```
def init_weights(m):  
    try:  
        m.reset_parameters()  
    except:  
        pass  
  
model = BertModel.from_pretrained(model_version)  
model = model.apply(init_weights)  
  
sentence_a = "I am happy"  
sentence_b = "I am not sad"  
show(model, model_type, tokenizer, sentence_a, sentence_b,  
display_mode='dark', layer=2, head=0)
```

Questions: What differences do you notice in the attention patterns between the randomly initialized and trained BERT models?
Run the final cell in the notebook. What are some words or tokens that you would expect strong attention between? What might you guess about the gradients of this attention head for those words?

Excellent! This is a **critical experiment** - comparing **randomly initialized BERT** vs **pre-trained BERT**. This reveals what pre-training actually teaches the model. Let me analyze what you should expect:

Dramatic Differences: Random vs Pre-trained BERT

Randomly Initialized BERT (Your Code):

Expected Attention Patterns:

1. Uniform/Chaotic Attention:

- Attention will be nearly **random and uniform**
- Each token attends to all other tokens with similar weights

Correct

- No clear patterns or structure

2. No Linguistic Structure:

- No subject-verb relationships
- No semantic connections
- No syntactic parsing

3. Special Token Confusion:

- [CLS] and [SEP] won't play special roles
- They'll be treated like any other token

4. Example for "I am happy" / "I am not sad":

- "happy" might attend randomly to "I", "am", [CLS] , etc.
- "not" won't properly negate "sad"
- The antonym relationship (happy ↔ sad) won't be captured

Pre-trained BERT (Normal):

Expected Attention Patterns:

1. Structured, Meaningful Attention:

- Clear linguistic patterns
- "am" → "happy" (verb-adjective)
- "not" → "sad" (negation)

2. Semantic Relationships:

- "happy" and "sad" might attend to each other (antonyms)
- Both "I" tokens might connect (coreference)

3. Specialized Heads:

- Different heads capture different relationships
- Some heads for negation, some for adjectives, etc.

Specific Words with Strong Attention (in Pre-trained Model):

Expected Strong Attention Connections:

1. Negation Focus:

- "not" → "sad" (VERY strong attention)
- This is a critical linguistic pattern BERT learns

2. Subject-Verb-Adjective:

- "I" → "am" → "happy" chain
- "I" → "am" → "not" → "sad" chain

3. Antonym Relationship:

- "happy" ↔ "sad" (across sentences)
- Especially in later layers

4. Contrast Attention:

- "happy" might attend to "not sad" as a contrastive phrase

5. Self-Attention of "I":

- Both "I" tokens attending to each other
- Building discourse coherence

6. Verb Consistency:

- Both "am" tokens might attend to each other

Gradient Analysis for Attention Heads:

What Gradients Tell Us:

For strong attention connections like "not" → "sad":

1. Large Gradients During Training:

Most important part

- Initially: Random attention → large error → large gradient
- The model learns "negation should focus on the negated word"
- Gradient signal pushes attention weights to specialize

2. Gradient Flow Pattern:

text

Loss → Attention weights → Q/K/V projections → Embeddings

- Gradients flow backward through attention mechanism
- Strongest gradients for frequently occurring patterns

3. Specific Predictions for "not" → "sad":

- **Query from "not"**: Learns to ask "what am I negating?"
- **Key from "sad"**: Learns to signal "I'm a negatable adjective"
- **Value from "sad"**: When attended to by "not", provides "negated meaning"

Why Gradients Matter for These Patterns:

1. Negation is High-Utility:

- Changes meaning dramatically ("happy" vs "not happy")
- Large semantic impact → large gradient signal
- Model quickly learns to focus attention here

2. Antonyms Create Contrastive Learning:

- "happy" vs "sad" appear in similar contexts
- Model learns to distinguish them through attention
- Gradient contrast helps separate these representations

3. Function Words Guide Attention:

- Words like "not", "am", "the" become **attention routers**
- They learn to query for specific types of information
- Their gradients teach them how to direct attention

Visual Differences You'll See:

Random BERT:

- **Color uniformity**: All attention lines similar intensity

- **No clear patterns:** Random web of connections
- **No head specialization:** All heads look similar
- **Poor [CLS] integration:** No meaningful aggregation

Pre-trained BERT:

- **Color variation:** Some lines much darker (strong attention)
- **Clear patterns:** Linguistic structures visible
- **Head diversity:** Different heads show different patterns
- **Effective [CLS] use:** Integrates information meaningfully

The Training Process (What Gradients Enable):

1. **Phase 1: Random initialization** → chaotic attention
2. **Phase 2: Masked language modeling** → gradients teach attention
3. **Phase 3: Specialization** → heads learn specific functions

For "not" → "sad" specifically:

- **Initial random state:** "not" attends randomly to all tokens
- **During MLM training:** When "sad" is masked, model needs "not" to predict it
- **Gradient signal:** Teaches attention to focus "not" on following adjective
- **Final state:** Strong, specialized attention pattern

What This Experiment Reveals:

1. **Pre-training isn't just better weights** - it's **learned attention patterns**
2. **Attention mechanisms need training** to become linguistically meaningful
3. **Gradients drive specialization** of different attention heads
4. **Frequent linguistic patterns** get strong attention connections

Key Insight from This Comparison: